

# Web and Data Engineering

---

## Kapitel 02: Architektur von Web-Anwendungen

Prof. Dr. Michael Granitzer

24 März 2026

## Speaker notes

Architektur gibt Antworten auf die in Vorlesung 01 beschriebene Komplexität: Wie werden einzelne Rechner zu Systemen? Wie lässt sich Skalierung von bloßem Hardware-Zuwachs trennen? Welche Entscheidungen legen die Grenzen späterer Änderungen fest? Diese Vorlesung beantwortet diese Fragen strukturiert — von Ressourcen über Prinzipien bis zu konkreten Architekturstilen.



# Ziel dieser Vorlesung

Architektur bestimmt, wie Web-Systeme strukturiert, verteilt und betrieben werden. Diese Vorlesung klärt, **welche Architekturentscheidungen** typisch sind, **warum** sie getroffen werden und **wie** sie sich auf Wartbarkeit, Skalierung und Datenverarbeitung auswirken.

## Leitfragen

- Was verwaltet Software eigentlich — und warum reicht ein Rechner nicht?
- Warum ist Architektur mehr als ein Implementierungsdetail?
- Welche Architekturstile gibt es — und wie funktionieren sie?
- Wie strukturiert das 3-Schichten-Modell Web-Anwendungen?
- Wie hängen Client-Typen, Deployment und Kopplung zusammen?
- Welche Architekturen braucht die Datenverarbeitung in Web-Systemen?

Die Reihenfolge der Themen ist nicht zufällig: Hardware-Ressourcen und ihre Grenzen motivieren verteilte Systeme; Architekturprinzipien bilden das konzeptionelle Werkzeug für Entwurfsentscheidungen; konkrete Stile wie Monolith, Event-Driven und Microservices sind Antworten auf unterschiedliche Skalierungs- und Teamanforderungen; Datenarchitekturen schließen den Kreis, weil Persistenz- und Verarbeitungsstrategie maßgeblich über Skalierbarkeit und Konsistenz entscheiden.

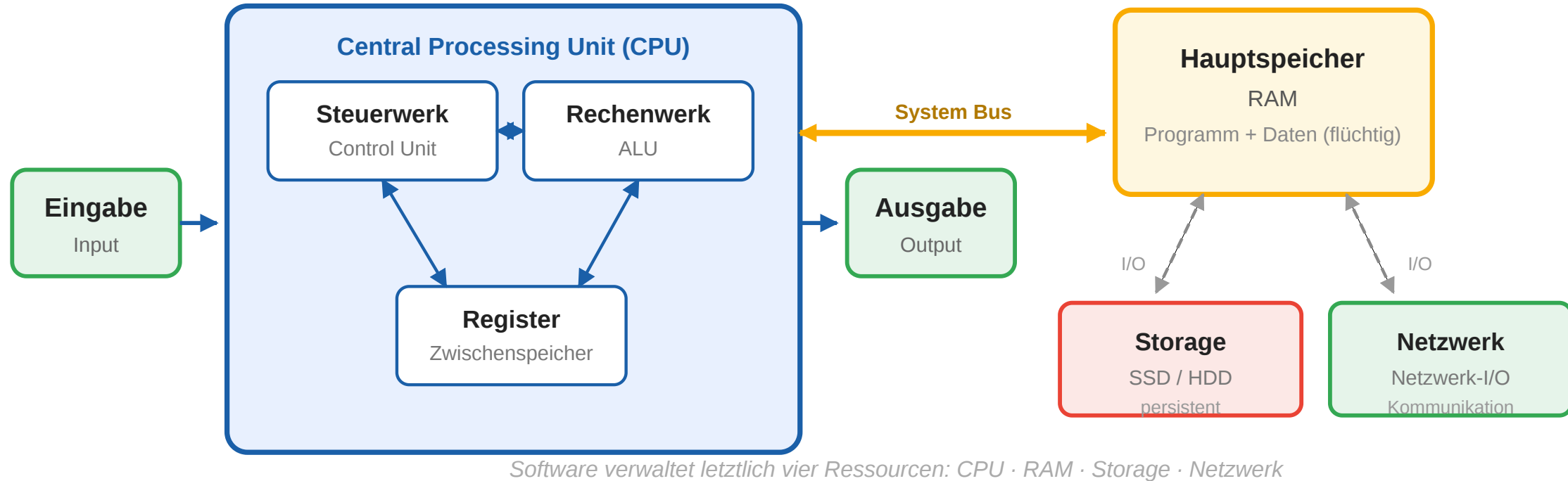
# Rechnerarchitektur und Ressourcen

---

**Leitfrage:** Was verwaltet Software eigentlich — und warum reicht ein einzelner Rechner für Web-Systeme meist nicht aus?

# Die Von-Neumann-Architektur

Nahezu alle heutigen Computer folgen dem **Von-Neumann-Modell** (1945): ein gemeinsamer Speicher für Programm und Daten, eine Verarbeitungseinheit und ein sequentieller Befehlszyklus.

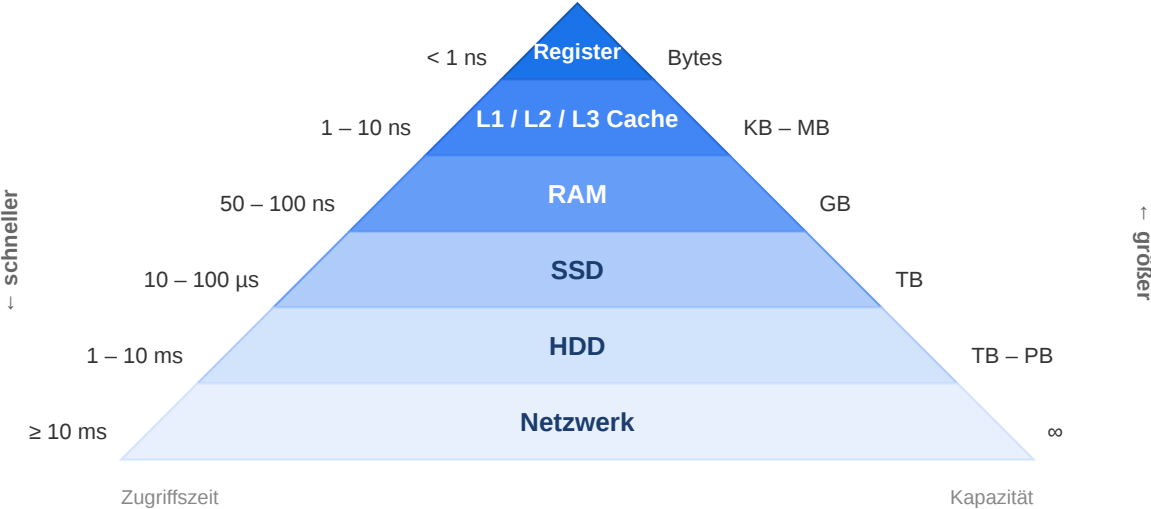


Software verwaltet letztlich vier physische Ressourcen: **Rechenleistung** (CPU), **Arbeitsspeicher** (RAM), **persistenter Speicher** (Storage) und **Netzwerk** (I/O). Jede Architekturentscheidung betrifft mindestens eine davon.

Hier liegt die Basiserkenntnis: Architektur ist keine Abstraktion ohne Materie. Jede Softwareentscheidung wirkt auf reale Ressourcen und damit auf Kosten, Latenz und Ausfallsicherheit.

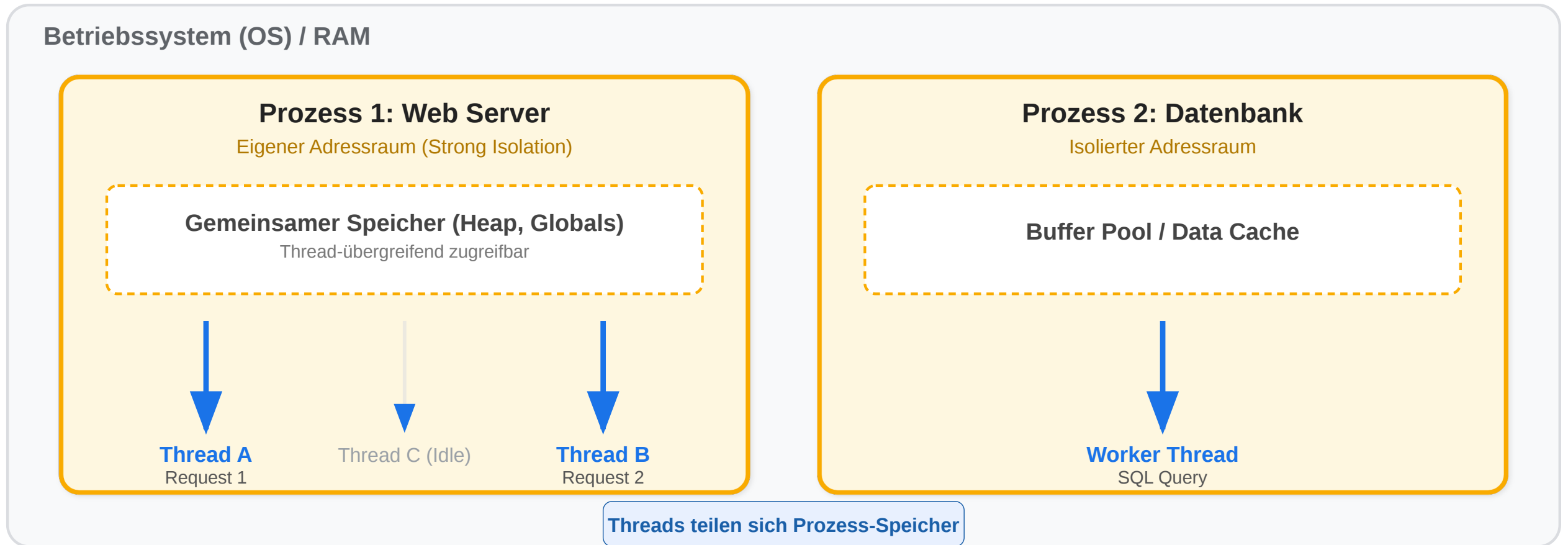
# Ressourcen eines Rechners

| Ressource | Funktion              | Typisch (Server) | Engpass-Symptom                  |
|-----------|-----------------------|------------------|----------------------------------|
| CPU       | Berechnung, Steuerung | 8–128 Kerne      | hohe Latenz, lange Antwortzeiten |
| RAM       | aktive Daten          | 32–512 GB        | Out-of-Memory, Swapping          |
| Storage   | persistente Daten     | TB bis PB        | langsame Reads/Writes            |
| Netzwerk  | Kommunikation         | 1–100 Gbit/s     | Timeouts, Paketverlust           |





# Prozesse und Threads



- **Prozess:** Eine laufende Instanz eines Programms mit eigenem, isoliertem Speicherbereich.
- **Thread:** Ein leichtgewichtiger Ausführungsstrang innerhalb eines Prozesses, der sich den Speicher mit anderen Threads desselben Prozesses teilt.

# Prozess vs. Thread

|           | Prozess            | Thread                 |
|-----------|--------------------|------------------------|
| Speicher  | eigener Adressraum | teilt Prozess-Speicher |
| Isolation | stark              | schwach                |
| Erzeugung | aufwändig          | leichtgewichtig        |

Ein Web-Server bedient viele gleichzeitige Requests. Ob er Threads, Prozesse oder asynchrone Events nutzt, ist eine zentrale Designentscheidung — bestimmt Durchsatz und Ressourcenverbrauch.

Diese Gegenüberstellung macht klar, dass Parallelität nicht kostenlos ist. Die Wahl des Ausführungsmodells ist bereits Architektur.

# Warum ein Rechner nicht reicht

## Grenzen einzelner Rechner (Faustregeln)

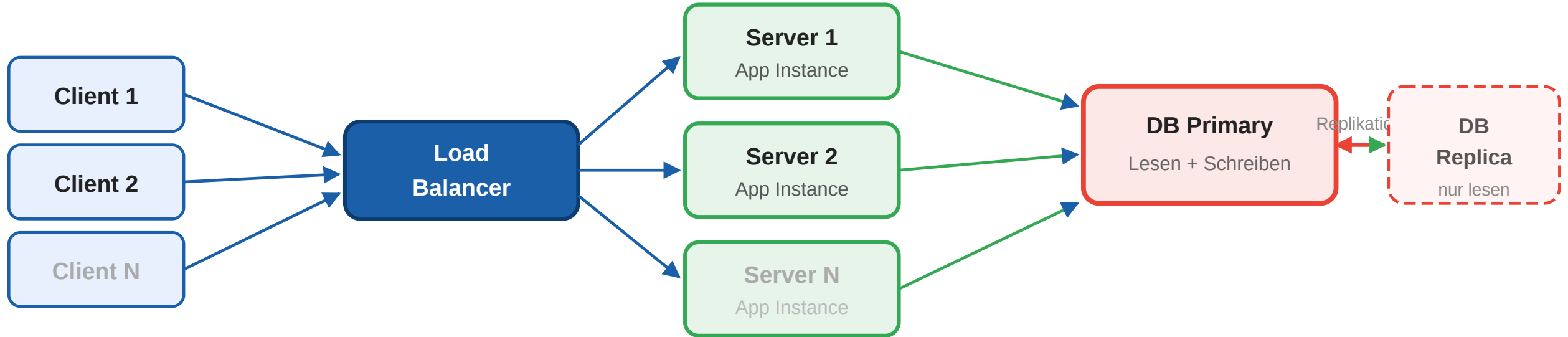
- **CPU:** Einfache HTTP-Requests: Tausende/s — rechenintensive Operationen (ML, Bildverarbeitung) deutlich weniger
- **RAM:**  $100.000 \text{ Sessions} \times 1 \text{ MB} = 100 \text{ GB}$  — Sessiongröße ist entscheidend
- **Storage I/O:** SSD: 10.000–100.000 IOPS — komplexe DB-Queries können das schnell ausschöpfen
- **Netzwerk:**  $10 \text{ Gbit/s} = 1,25 \text{ GB/s}$  — bei 10.000 Nutzern  $\times 1 \text{ Mbit/s}$  bereits voll ausgelastet

## Skalierungsstrategien

| Strategie            | Beschreibung                          | Grenze                               |
|----------------------|---------------------------------------|--------------------------------------|
| Vertikal skalieren   | Mehr RAM, schnellere CPU, größere SSD | Physische und ökonomische Obergrenze |
| Horizontal skalieren | Mehr Server, Last verteilen           | Koordination, Konsistenz, Netzwerk   |

Konkrete Zahlen sind immer Faustregeln und stark workload-abhängig. Ziel ist das Verständnis der Größenordnungen und Engpass-Typen, nicht exakte Benchmarks.

# Vom einzelnen Rechner zum verteilten System



*Neue Herausforderungen: Netzwerklatenz · Partielle Ausfälle · Datenkonsistenz · Koordination*

## Neue Herausforderungen

- **Netzwerk-Latenz** — ms statt ns (RAM)
- **Partielle Ausfälle** — einzelne Server fallen aus
- **Konsistenz von Software** — welche Version ist aktuell?
- **Koordination von Prozessen** — wer bearbeitet was?

Es entstehen neue Architekturentscheidungen

Die Botschaft ist: Ein verteiltes System ist keine Eleganzübung, sondern die Konsequenz aus Ressourcen- und Skalierungsgrenzen.

# Takeaway

Software-Architektur verwaltet physische Ressourcen: CPU, RAM, Storage und Netzwerk. Ein einzelner Rechner hat endliche Kapazität — sobald diese nicht mehr ausreicht, entsteht ein verteiltes System mit neuen Herausforderungen (Latenz, Ausfälle, Konsistenz).

## Schlüsselbegriffe

| Begriff                 | Erklärung                                                             |
|-------------------------|-----------------------------------------------------------------------|
| Von-Neumann-Architektur | Rechnermodell mit gemeinsamem Speicher für Programm und Daten         |
| Prozess                 | Laufende Instanz eines Programms mit eigenem Speicherbereich          |
| Thread                  | Leichtgewichtiger Ausführungsstrang innerhalb eines Prozesses         |
| Concurrency             | Mehrere Aufgaben machen Fortschritt — durch Scheduling auf einem Kern |
| Parallelismus           | Mehrere Aufgaben laufen gleichzeitig auf verschiedenen Kernen         |
| Vertikale Skalierung    | Leistungsstärkere Hardware für einen Server (Scale Up)                |
| Horizontale Skalierung  | Last auf mehrere Server verteilen (Scale Out)                         |



Vertikale Skalierung (stärkere Hardware) stößt physisch an Grenzen und erzeugt Single Points of Failure. Horizontale Skalierung (mehr Instanzen) erfordert zustandslose Komponenten, Load Balancing und verteilte Datenhaltung — sie ist aufwändiger, aber unbegrenzt skalierbar. In der Praxis werden beide kombiniert: vertikale Skalierung für die Datenbankschicht, horizontale für die Applikationsschicht.

# Software-Architekturen

---

**Leitfrage:** Warum lohnt es sich, eine Web-Anwendung architektonisch zu zerlegen, bevor über Frameworks oder Programmiersprachen entschieden wird?

# Was ist Software-Architektur?

*„The software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.”*

*— Bass, Clements & Kazman, Software Architecture in Practice (4th ed.)*

Software-Architektur beschreibt die **oberste Struktur** eines Systems — die schwer änderbaren Entscheidungen und die Beziehungen zwischen den wichtigsten Teilen.

Architekturentscheidungen sind diejenigen, deren nachträgliche Änderung unverhältnismäßig hohen Aufwand erzeugt — etwa die Wahl des Datenbankmodells, des Kommunikationsmusters zwischen Komponenten oder der Deployment-Einheit. Implementierungsentscheidungen (Klassenname, Algorithmusdetail) sind dagegen lokal reversibel. Diese Asymmetrie erklärt, warum Architekturarbeit früh im Entwicklungsprozess den höchsten Hebel hat.

# Typische Ziele

- Komplexität reduzieren
- Verantwortlichkeiten trennen
- Schnittstellen stabilisieren
- Wartung und Weiterentwicklung vorbereiten
- Sicherstellen von
  - Skalierbarkeit
  - Testbarkeit
  - Zuverlässigkeit
  - Sicherheit
  - Performance
  - ...

Die Liste ist bewusst breit. Architektur ist immer ein Kompromiss zwischen Qualitätszielen, nie nur ein einzelnes Kriterium.

# Architekturentscheidung vs. Implementierung

|                 | Architekturentscheidung                        | Implementierungsentscheidung      |
|-----------------|------------------------------------------------|-----------------------------------|
| Änderungskosten | hoch — betrifft viele Komponenten              | gering — lokal begrenzt           |
| Beispiel Web    | „Monolith oder Microservices?“                 | „Welches ORM nutzen wir?“         |
| Beispiel Web    | „REST oder GraphQL als API-Stil?“              | „Welche HTTP-Library?“            |
| Beispiel Web    | „Wo liegt Session-State — Client oder Server?“ | „Redis oder Memcached als Cache?“ |
| Lebensdauer     | Jahre — schwer revidierbar                     | Wochen bis Monate — austauschbar  |

**Faustregel:** Wenn eine Entscheidung **teuer zu revidieren** ist und **viele Teile des Systems betrifft**, ist sie architektonisch.

Diese Faustregel ist praktisch wichtiger als jede Definition. Sie hilft bei der täglichen Unterscheidung zwischen "Design" und "Implementierung".



# Architektonische Prinzipien

| Prinzip                        | Kernaussage                                                  | Schlecht                                                         | Gut                                                       |
|--------------------------------|--------------------------------------------------------------|------------------------------------------------------------------|-----------------------------------------------------------|
| Separation of Concerns         | Jede Komponente hat eine klar abgegrenzte Verantwortung      | <code>&lt;p&gt;&lt;?= \$db-&gt;query(...) ?&gt;&lt;/p&gt;</code> | Controller → Service → Repository → DB                    |
| Single Responsibility          | Ein Modul tut genau eine Sache                               | OrderService speichert + mailt + berechnet Steuer                | OrderService + EmailService + TaxService                  |
| Principle of Least Knowledge   | Komponenten kennen nur die Schnittstelle — nicht die Interna | Service A liest direkt von DB Daten aus Service B                | Service A ruft Service B per API auf                      |
| Build to Change                | Architektur für Änderungen entwerfen, nicht für Stabilität   | Interne Felder direkt exponiert, kein Versionierungskonzept      | API-Versionierung, Feature Flags                          |
| DRY - Dont Repeat Yourself     | Logik nicht duplizieren                                      | Validierung separat in Client und Server                         | Validierung zentral — einmal definiert, überall verwendet |
| YAGNI - You Aint Gonna Need it | Nur bauen, was jetzt gebraucht wird                          | Microservice-Setup für ein Drei-Personen-Team                    | Monolith starten, erst bei echtem Bedarf aufteilen        |
| KISS - Keep It Simple, Stupid  | Komplexität vermeiden                                        | Microservice-Setup für ein Drei-Personen-Team                    | Monolith starten, erst bei echtem Bedarf aufteilen        |

Die Prinzipien sind keine Dogmen. Sie sind Abkürzungen für wiederkehrende Erfahrungen, die helfen, Wartbarkeit und Änderbarkeit aktiv zu schützen.

# Warum Architektur für Web-Systeme zählt

Web-Anwendungen kombinieren viele Technologien und Verantwortlichkeiten in einem System. Ohne explizite Architektur wachsen Abhängigkeiten unkontrolliert.

| Situation           | Ohne Architektur                                           | Mit Architektur                                      |
|---------------------|------------------------------------------------------------|------------------------------------------------------|
| Datenbank wechseln  | Änderungen in hunderten Dateien, da SQL direkt im Code     | Data Access Layer austauschen, Rest bleibt stabil    |
| Mobile App ergänzen | Backend liefert HTML — App braucht JSON-API → großer Umbau | API existiert bereits, Mobile Client wird angebunden |
| Team wächst         | Alle arbeiten im selben Code, ständige Konflikte           | Klare Modulgrenze, Teams arbeiten unabhängig         |
| Performance-Problem | Unklar, welche Komponente das Problem verursacht           | Monitoring pro Schicht, gezielte Skalierung möglich  |

# Typische Architekturfragen

Bevor Code geschrieben wird, sollten diese Fragen beantwortet sein:

- Welche Teile dürfen direkt voneinander abhängen?
- Wo liegt Geschäftslogik — im Client, im Server, in einem separaten Service?
- Welche Entscheidungen müssen lange stabil bleiben?
- Wie wird das System deployt — als Monolith, als Container, als serverless Functions?
- Welche Querschnittsthemen (Sicherheit, Logging, Monitoring) betreffen mehrere Komponenten?
- Wie kommunizieren die Teile — synchron per API oder asynchron per Events?

Architektur ist keine Zusatzdokumentation. Sie ist das Mittel, um ein komplexes Web-System überhaupt beherrschbar zu machen.

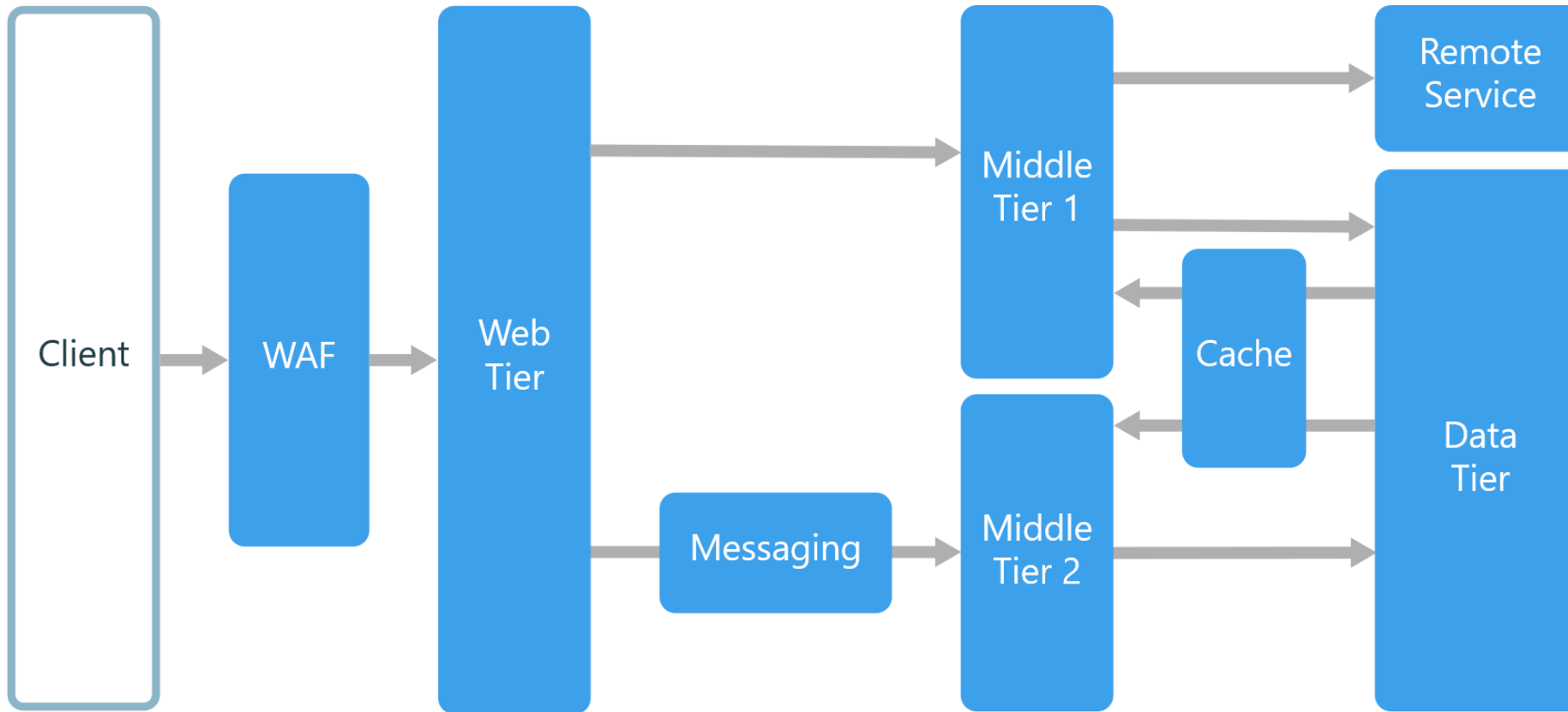
Der Punkt ist nicht Dokumentationspflege, sondern Steuerbarkeit. Architektur ist eine Arbeitsgrundlage für Entwicklung und Betrieb.

# Architekturstile für Web-Anwendungen

---

**Leitfrage:** Welcher Architekturstil passt zu einer Web-Anwendung, wenn sich Anforderungen, Lastprofil und Teamstruktur unterscheiden?

# N-Tier als Referenzstil

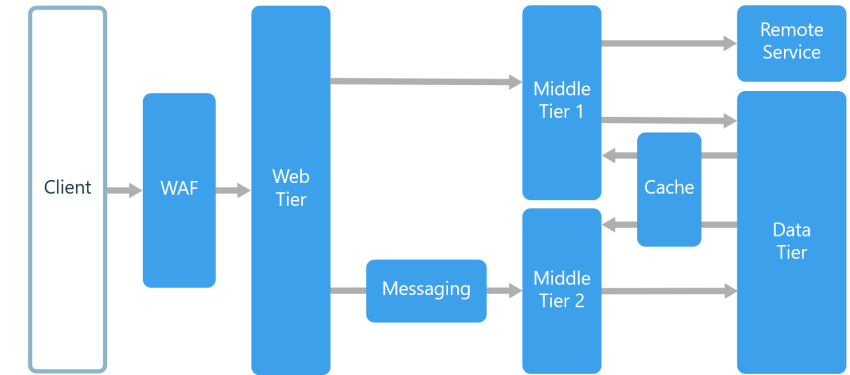


- Einteilung in Schichten mit klaren Schnittstellen und Vereinbarungen
- eine Schicht kann nur auf die Schicht davor zugreifen (Closed Layer) oder auf alle darunter (Open Layer)
- Schichten können logische Unterteilung (Layers) oder physische Verteilung (Tiers) darstellen
- **Anwendungsbereiche:** Standard-Webanwendungen, heterogene Systeme

# N-Tier: Wie funktioniert das?

Ein Request durchläuft die Schichten sequentiell — die Response traversiert sie in umgekehrter Reihenfolge zurück:

1. **Client** → HTTP-Request an **Web Tier**
2. **Web Tier** → Auth, Routing, Weitergabe an **Middle Tier**
3. **Middle Tier** → Geschäftslogik, Zugriff auf **Data Tier**
4. **Data Tier** → Daten lesen/schreiben, Ergebnis zurück
5. **Middle Tier** → verarbeitetes Ergebnis an **Web Tier**
6. **Web Tier** → HTTP-Response an **Client**



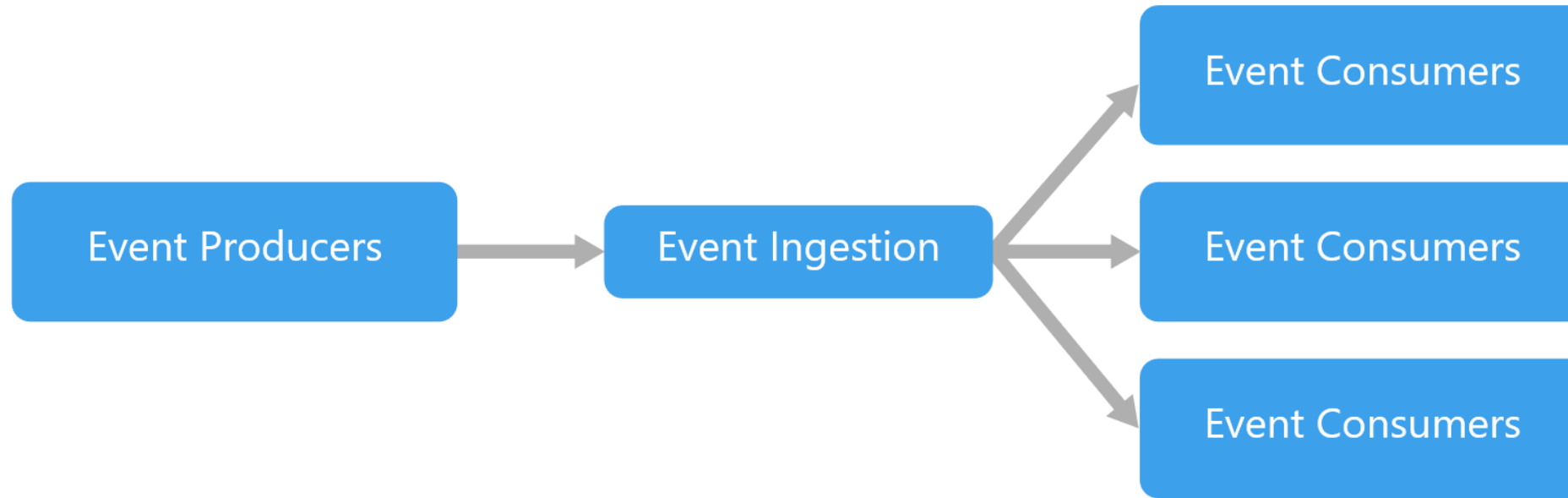
**Schlüsselprinzip:** Jede Schicht kennt nur die Schnittstelle der nächsten — nicht deren Implementierung. Schichten sind unabhängig austauschbar und skalierbar.

**Herausforderungen:** Querschnittsthemen (Logging, Sicherheit) sind aufwändig; Änderungen betreffen oft mehrere Schichten gleichzeitig.



Das Referenzmodell ist bewusst simpel, weil es Kommunikationsregeln sichtbar macht. Genau diese Regeln sind in großen Web-Systemen der eigentliche Gewinn.

# Event-Driven Architecture

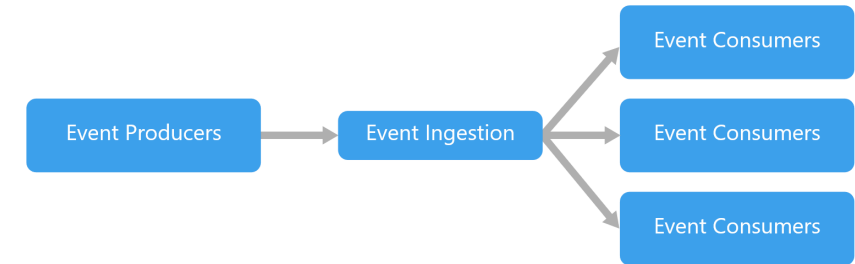


Komponenten kommunizieren nicht direkt, sondern über **Events** — asynchrone Nachrichten, die eine Zustandsänderung beschreiben.

- **Producer** erzeugt ein Event (z.B. „Bestellung aufgegeben“)
- **Broker** (z.B. Kafka, RabbitMQ) nimmt das Event entgegen und verteilt es
- **Consumer** reagiert auf das Event unabhängig vom Producer

# Event-Driven: Beispiel Bestellprozess

1. **Bestellservice** → OrderPlaced an Broker
2. Broker verteilt an **Zahlungsservice**
3. Zahlungsservice verarbeitet Zahlung → PaymentCompleted an Broker
4. Broker verteilt an **Lagerservice**
5. Lagerservice reserviert Ware



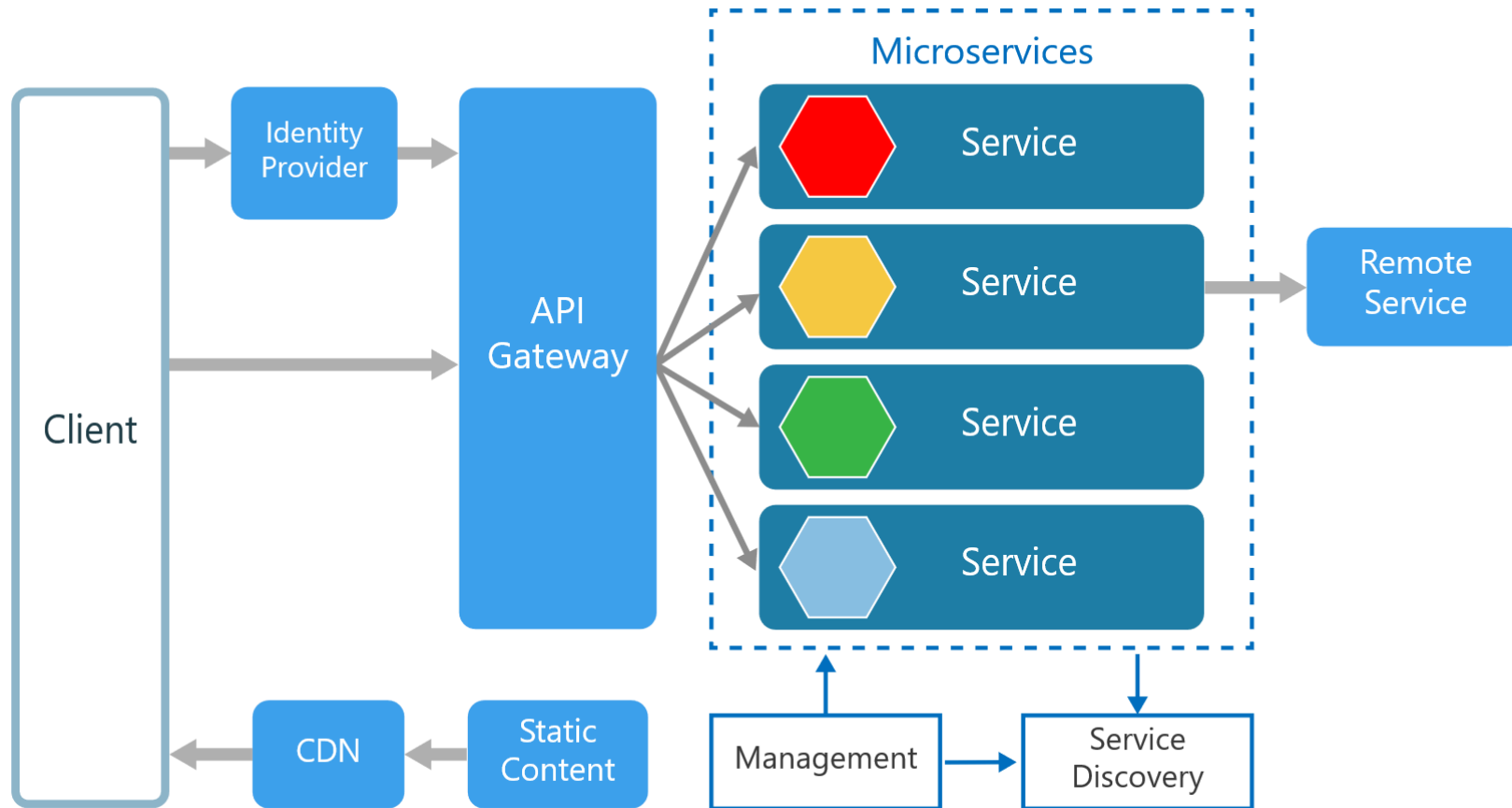
**Schlüsselprinzip:** Neue Consumer können hinzugefügt werden, ohne bestehende Komponenten zu ändern → hohe Entkopplung.

## Herausforderungen:

- Reihenfolge nicht automatisch garantiert
- Fehlerbehandlung komplexer (Consumer-Ausfall?)
- Debugging über asynchrone Schritte schwieriger

Die Funktion des Event-Modells ist Entkopplung. Gleichzeitig zeigen die Nachteile, dass Asynchronität vor allem Betriebs- und Debugging-Arbeit verschiebt.

# Microservices

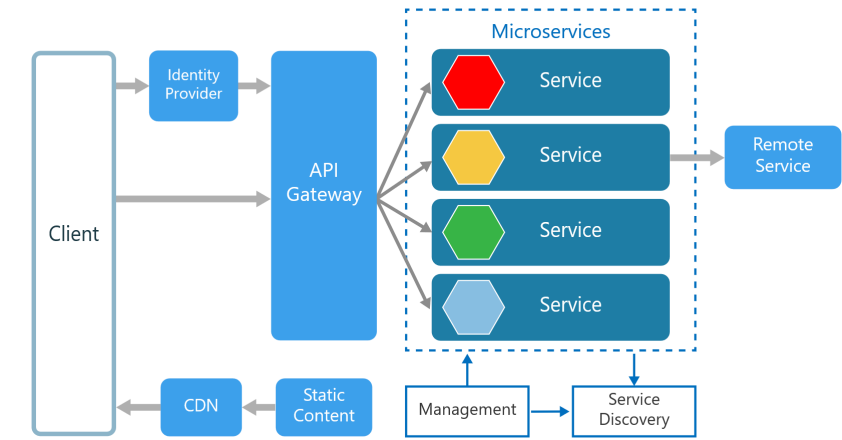


Das System wird in **kleine, autonome Dienste** zerlegt — jeder mit abgegrenzter fachlicher Verantwortung und unabhängigem Deployment.

- jeder Service hat eigene Datenbank, eigenes Deployment, eigenes Team
- Kommunikation über REST-APIs, gRPC oder Events
- ein **API Gateway** routet externe Anfragen an die richtigen Services

# Microservices: Beispiel Homepage-Request

1. **Client** → GET /homepage an **API Gateway**
2. Gateway ruft parallel auf:
  - User Service → Profildaten
  - Recommendations Service → Empfehlungen
  - Content Service → Katalogdaten
3. Gateway komponiert die Antworten → zusammengesetzte Response an Client



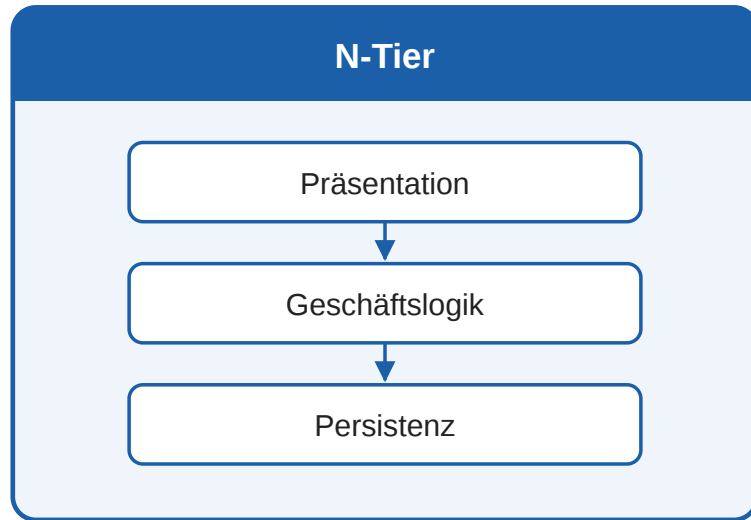
**Schlüsselprinzip:** Jeder Service ist autonom — eigene Sprache, eigene DB, eigener Release-Zyklus. Teams arbeiten unabhängig.

## Herausforderungen:

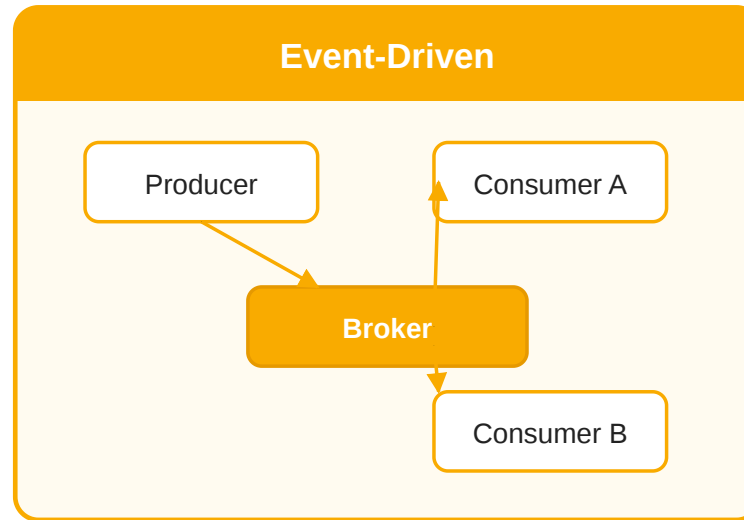
- Netzwerklatenz, partielle Ausfälle
- Jeder Service braucht Monitoring + Deployment-Pipeline
- Service Discovery & Governance

Microservices sind eine Organisations- und Betriebsentscheidung ebenso sehr wie eine technische. Der Gewinn ist Teamautonomie, der Preis ist Koordination.

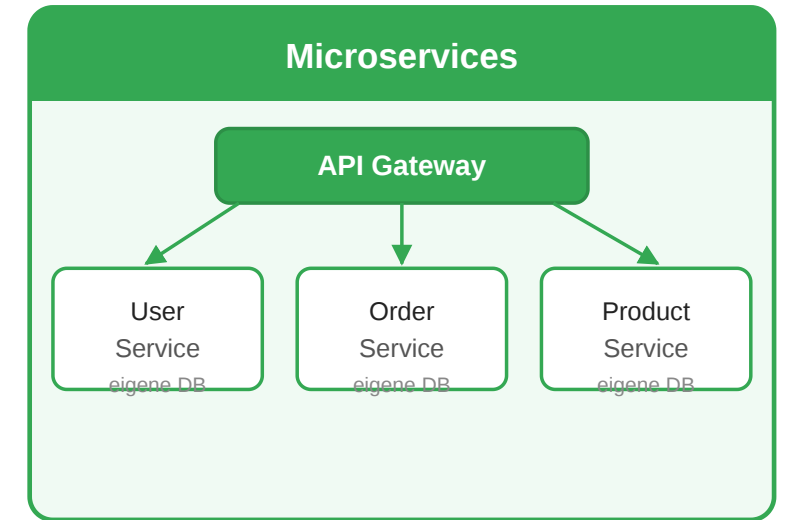
# Architekturstile im Vergleich



synchron · sequentiell  
klassische Web-Apps



asynchron · locker gekoppelt  
Streaming · Pipelines



autonom · unabhängig deploybar  
große, evolvierende Plattformen

Es gibt keinen universell besten Architekturstil. Die Wahl hängt von Domäne, Lastprofil, Änderungsrate und organisatorischem Setup ab. In der Praxis kombinieren viele Systeme mehrere Stile — z.B. N-Tier für die Kernapplikation mit Event-Driven für asynchrone Verarbeitung.



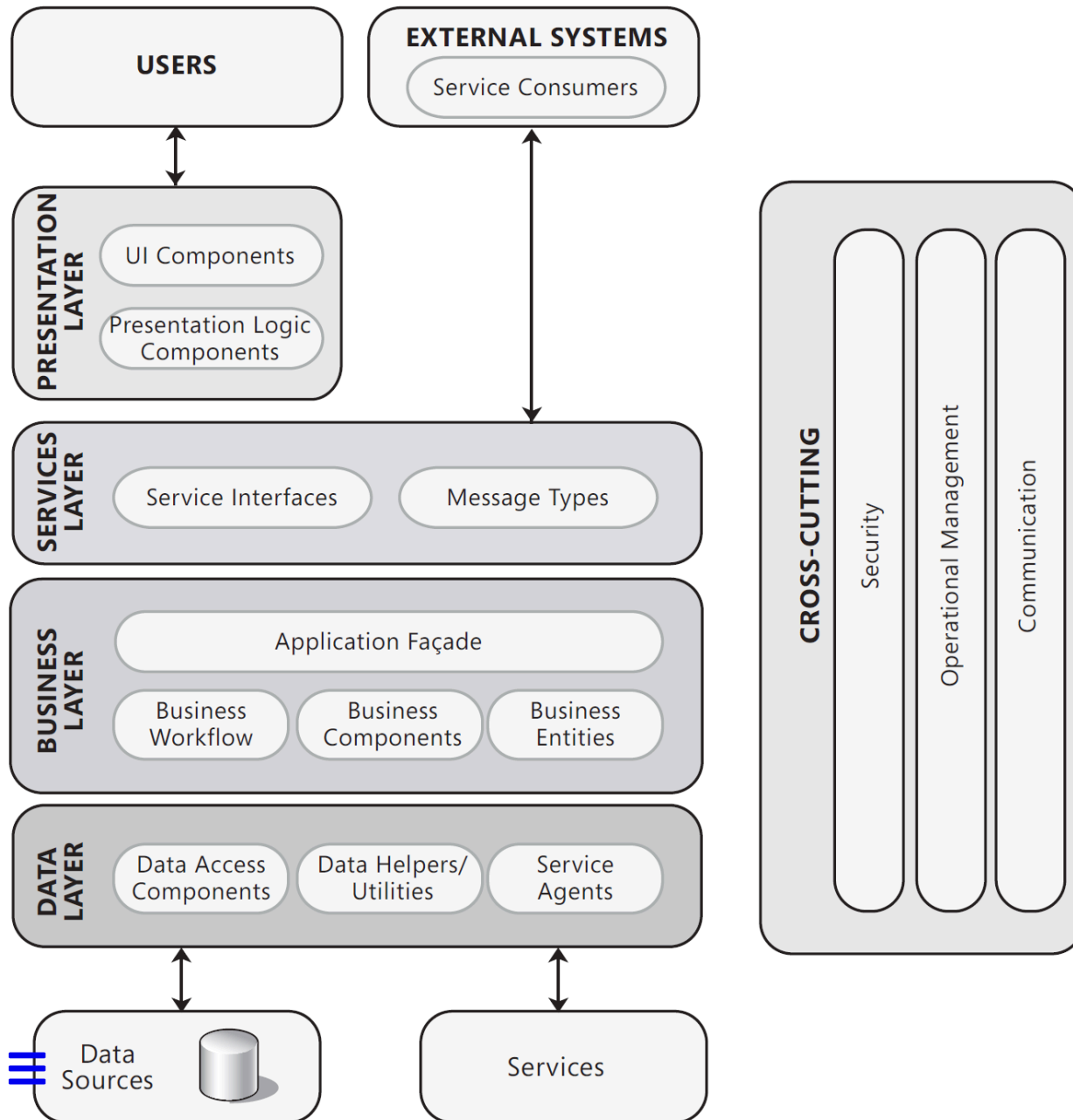
Reife Produktionssysteme kombinieren typischerweise mehrere Stile: ein monolithischer Kern für die domänenzentrale Logik, Event-Driven für asynchrone Abläufe wie Benachrichtigungen und Berichte, einzelne eigenständige Services dort, wo Teams hohe Deployment-Autonomie benötigen. Die Entscheidung zwischen Stilen hängt nicht von Präferenzen ab, sondern von Team-Größe, Änderungsfrequenz, Latenzanforderungen und Betriebskomplexität — Kriterien, die sich im Projektverlauf verschieben können.

# Logisches 3-Schichten-Modell

---

**Leitfrage:** Wie lässt sich eine Web-Anwendung so strukturieren, dass Benutzeroberfläche, Logik und Datenhaltung getrennt, aber koordiniert bleiben?

# Logisches 3-Schichten-Modell



Das klassische 3-Schichten-Modell teilt Anwendungssysteme in drei Schichten:

| Schicht      | Aufgabe                     |
|--------------|-----------------------------|
| Präsentation | UI und Interaktion          |
| Anwendung    | Geschäftslogik & Regeln     |
| Persistenz   | dauerhafte Datenspeicherung |

Die Anwendungsschicht kann in Dienst- und Logikschicht unterteilt werden. Dienste kapseln die Geschäftslogik und stellen Schnittstellen für die Präsentationsschicht bereit. **Layer = logische Trennung.** Mehrere Schichten können im selben Prozess laufen — physische Verteilung folgt erst im nächsten Schritt.



# Was gehört in welche Schicht?

**Aufgabe:** Ein Nutzer kauft im Online-Shop ein Produkt. Ordnen Sie diese Code-Fragmente den drei Schichten zu:

```
// (A)
<button onclick="order()">Kaufen</button>
<p>Preis: € 79,99</p>

// (B)
if (stock < 1) throw new OutOfStockException();
double total = price * quantity * taxRate;
sendConfirmationEmail(user);

// (C)
SELECT * FROM products WHERE id = 42;
INSERT INTO orders (user_id, product_id) ...
```

# Lösung

| Code                             | Schicht      | Warum                      |
|----------------------------------|--------------|----------------------------|
| (A) HTML/Button                  | Präsentation | Darstellung, Nutzereingabe |
| (B) Lagerprüfung, Steuer, E-Mail | Anwendung    | Geschäftsregeln            |
| (C) SQL-Abfragen                 | Persistenz   | Datenzugriff               |

**Faustregel:** Ändert sich etwas, wenn die Datenbank ausgetauscht wird? → Persistenz. Wenn das Design geändert wird? → Präsentation. Alles dazwischen → Anwendungsschicht.

# Warum die Trennung praktisch wichtig ist

## Ohne Schichtentrennung

```
// Alles gemischt im Controller:  
String html = "<h1>Produkt</h1>";  
ResultSet rs = db.query("SELECT ...");  
if (rs.getInt("stock") > 0) {  
    html += "<button>Kaufen</button>";  
    double tax = rs.getDouble("price") * 0.19;  
    html += "<p>€ " + tax + "</p>";  
}  
out.print(html);
```

- Datenbank wechseln → ganzen Code anfassen
- Design ändern → Geschäftslogik anfassen
- Testen ohne Datenbank → unmöglich

## Mit Schichtentrennung

- **Persistenzschicht** liefert Product-Objekt
  - **Anwendungsschicht** berechnet Preis, prüft Lager
  - **Präsentationsschicht** rendert HTML/JSON
- Datenbank austauschen: nur Persistenzschicht ändern
- Design ändern: nur Präsentation ändern
- Geschäftslogik testen: ohne DB, ohne Browser

Das 3-Schichten-Modell ist keine veraltete Standardlösung, sondern eine robuste Denkstruktur. Die entscheidende Frage bei jeder Code-Zeile: **In welcher Schicht lebe ich gerade?**

Eine häufige Verletzung des Schichtenmodells ist Datenbanklogik in der Präsentationsschicht (z.B. SQL-Aufrufe im Controller oder im Template) oder Geschäftslogik in der Datenzugriffsschicht (z.B. fachliche Berechnungen im ORM-Mapping). Beides macht Schichten nicht isoliert testbar und bindet Änderungen an mehrere Stellen gleichzeitig. Das Repository-Pattern und das Service-Layer-Pattern sind verbreitete Lösungen, um diese Grenzen strukturell durchzusetzen.



# 3-Tier und Web-Anwendungsformen

---

**Leitfrage:** Aus welchen Komponenten besteht eine moderne Web-Anwendung — vom Browser bis zur Datenbank — und wie arbeiten diese zusammen?

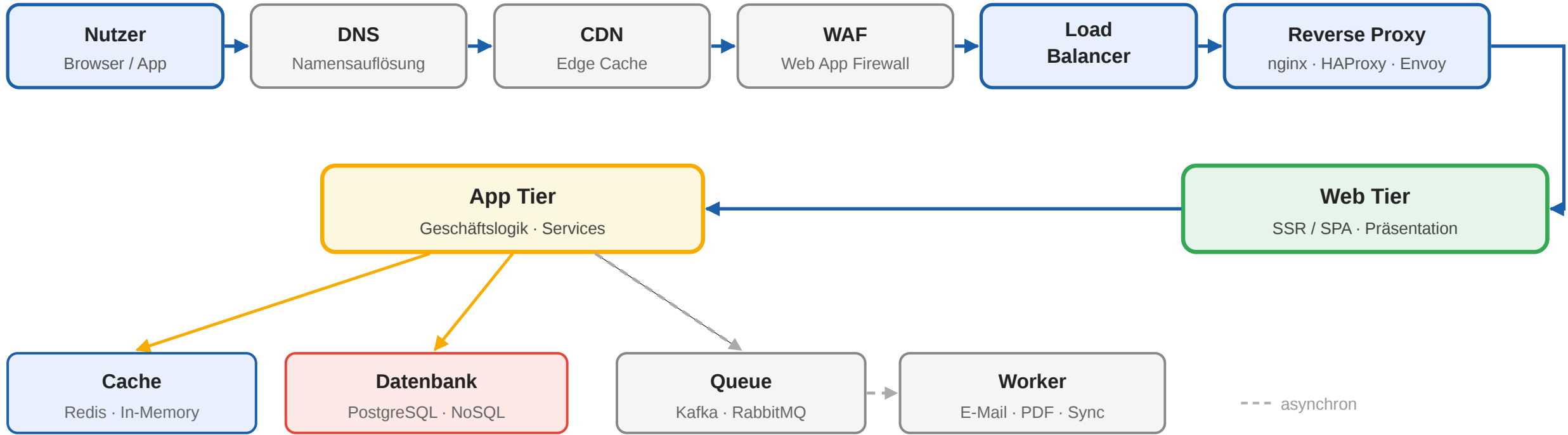
# Von Layer zu Tier

| Perspektive   | Layer                 | Tier                                    |
|---------------|-----------------------|-----------------------------------------|
| Trennung      | logisch               | physisch                                |
| Kommunikation | oft im selben Prozess | über Netz und Interprozesskommunikation |
| Hauptnutzen   | Strukturierung        | Verteilung, Skalierung, Isolation       |
| Zusatzkosten  | gering                | Deployment, Latenz, Koordination        |

**Layer** = logische Trennung (Architekturentscheidung). **Tier** = physische Trennung (Infrastrukturentscheidung).

# Anatomie einer modernen Web-Anwendung

Der Weg eines Requests vom Nutzer bis zur Datenbank durchläuft mehrere Infrastrukturkomponenten:



Jede dieser Komponenten erfüllt eine spezifische Rolle — die folgenden Slides erklären sie im Detail.

# Edge-Komponenten: Vor der Anwendung

Bevor ein Request die eigentliche Anwendung erreicht, durchläuft er mehrere Schutz- und Beschleunigungsschichten:

| Komponente  | Aufgabe                                                                                                     | Beispiele                         |
|-------------|-------------------------------------------------------------------------------------------------------------|-----------------------------------|
| DNS         | Auflösung des Domainnamens zur IP-Adresse; kann geografisches Routing ermöglichen                           | Route 53, Cloud DNS, Azure DNS    |
| CDN         | Cacht statische Assets (JS, CSS, Bilder) an Edge-Standorten nahe am Nutzer; reduziert Latenz und Serverlast | CloudFront, Cloud CDN, Azure CDN  |
| WAF         | Filtert bösartigen HTTP-Traffic; schützt gegen OWASP Top 10 (SQL Injection, XSS, etc.)                      | AWS WAF, Azure WAF, Cloud Armor   |
| DDoS-Schutz | Absorbiert volumetrische Angriffe bevor sie die Infrastruktur erreichen                                     | AWS Shield, Azure DDoS Protection |

Ohne Edge-Schutz ist die Anwendung direkt dem Internet ausgesetzt. In der Praxis sind CDN und WAF keine Extras, sondern Grundvoraussetzungen für Produktion.

# Traffic-Management: Load Balancer und API Gateway

| Komponente                   | Aufgabe                                                                                   | Beispiele                                   |
|------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------|
| <b>Load Balancer (L4/L7)</b> | Verteilt eingehende Requests auf mehrere Instanzen; Health Checks; SSL-Terminierung       | ALB, NLB, Cloud Load Balancing              |
| <b>API Gateway</b>           | Zentraler Einstiegspunkt für alle API-Anfragen; Routing, Authentifizierung, Rate Limiting | Kong, AWS API Gateway, Azure API Management |
| <b>Reverse Proxy</b>         | Leitet Client-Requests an Backend weiter; kann SSL, Kompression und Caching übernehmen    | Nginx, Envoy, Traefik                       |

**Schlüsselkonzept: Horizontale Skalierung:** Load Balancer ermöglichen **horizontale Skalierung** — statt einen Server größer zu machen (vertikal), werden mehrere gleichwertige Instanzen parallel betrieben. Fällt eine Instanz aus, übernehmen die anderen.

# Application Tier: Geschäftslogik

Die Anwendungsschicht enthält die eigentliche Fachlogik — Validierungen, Berechnungen, Orchestrierung von Diensten.

## Typische Komponenten

| Komponente             | Aufgabe                                                                    |
|------------------------|----------------------------------------------------------------------------|
| Application Façade     | Einheitlicher Einstiegspunkt für die Geschäftslogik; koordiniert Workflows |
| Business Components    | Fachliche Kernlogik (z.B. Preisberechnung, Bestellvalidierung)             |
| Business Entities      | Domänenobjekte, die fachliche Konzepte abbilden                            |
| Data Access Components | Abstrahieren den Zugriff auf die Persistenzschicht (Repository Pattern)    |
| Service Agents         | Kommunikation mit externen Diensten und APIs                               |

## Cross-Cutting Concerns

- **Security:** Authentifizierung, Autorisierung, Input-Validierung
- **Operational Management:** Logging, Monitoring, Health Checks
- **Communication:** Serialisierung, Protokollhandling, Fehlerbehandlung



# Asynchrone Verarbeitung: Message Queues

Nicht alle Aufgaben müssen synchron im Request-Zyklus erledigt werden. Message Queues entkoppeln zeitintensive Operationen:

## Ablauf

1. **Client** → POST /order an **App Tier**
2. App speichert Bestellung in DB, legt OrderCreated-Event in **Queue**
3. App antwortet sofort mit 202 Accepted — Client wartet nicht
4. **Worker** liest Event aus Queue, sendet E-Mail, generiert PDF, aktualisiert Lager

## Typische Einsatzfälle

- E-Mail-Versand, Push-Benachrichtigungen
- Bildverarbeitung, PDF-Generierung
- Synchronisation mit externen Systemen
- Langläufer: Reports, Datenexporte

## Technologien

- RabbitMQ · Apache Kafka
- AWS SQS · Azure Service Bus

# Caching und Data Tier

## Caching

| Ebene     | Aufgabe                         | Beispiele        |
|-----------|---------------------------------|------------------|
| CDN Cache | Statische Assets am Edge        | CloudFront       |
| In-Memory | Häufig gelesene Daten im RAM    | Redis, Memcached |
| App Cache | Sessions, berechnete Ergebnisse | Spring Cache     |

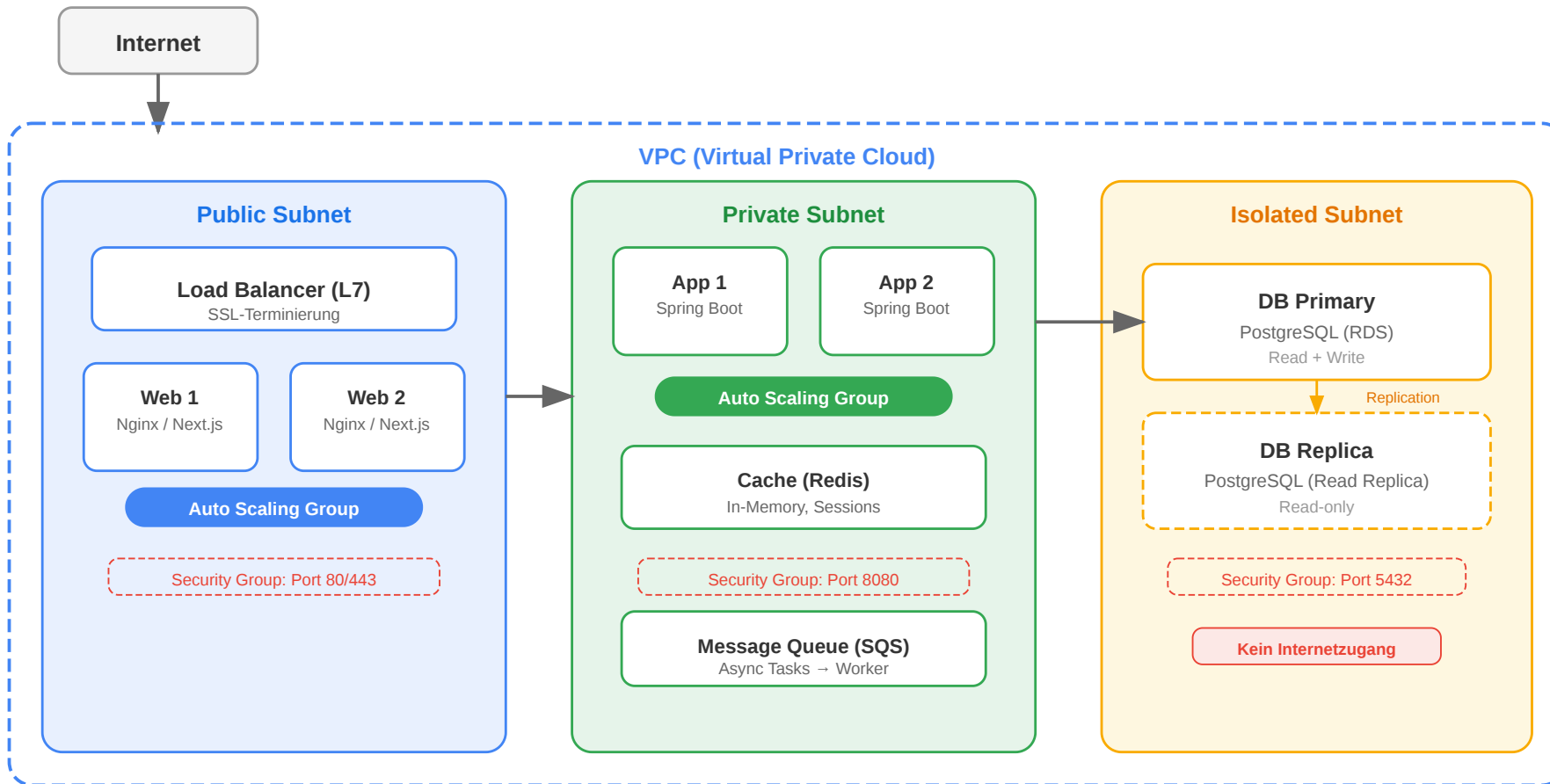
## Data Tier

| Typ            | Einsatz                     | Beispiele         |
|----------------|-----------------------------|-------------------|
| Relationale DB | Transaktionale Daten (ACID) | PostgreSQL, MySQL |
| NoSQL DB       | Flexible Schemata           | MongoDB, DynamoDB |
| Object Storage | Dateien, Bilder, Backups    | S3, Azure Blob    |
| Suchmaschine   | Volltextsuche               | Elasticsearch     |



# 3-Tier in der Cloud-Praxis

## 3-Tier in der Cloud



In der Cloud-Praxis wird jeder Tier in einem eigenen Subnetz isoliert:

- **Public Subnet:** Load Balancer, Web Tier
- **Private Subnet:** Application Tier, Cache
- **Isolated Subnet:** Datenbank (kein Internetzugang)
- **Security Groups** beschränken den Traffic zwischen Tiers
- **Auto Scaling** passt die Instanzanzahl pro Tier an die Last an

# Client-Varianten im Vergleich

Die Verteilung von Logik zwischen Client und Server bestimmt den Anwendungstyp:

| Variante            | Logik im Client                         | Logik am Server              | Typischer Einsatz                   |
|---------------------|-----------------------------------------|------------------------------|-------------------------------------|
| Thin Client         | minimal (nur Rendering)                 | UI-Erzeugung + Logik + Daten | Formularsysteme, Behörden-Apps      |
| Rich Client / SPA   | UI, Routing, Zustand                    | API + Logik + Daten          | Interaktive Plattformen, Dashboards |
| Mobile Client       | UI, Offline-Fähigkeit, lokaler Speicher | API + Logik + Daten + Sync   | Apps mit instabiler Konnektivität   |
| API-only (Headless) | beliebig (verschiedene Clients)         | nur API + Logik + Daten      | Multi-Client-Plattformen            |

Verschiedene Technologien je nach dem Grad der Logik im Client:

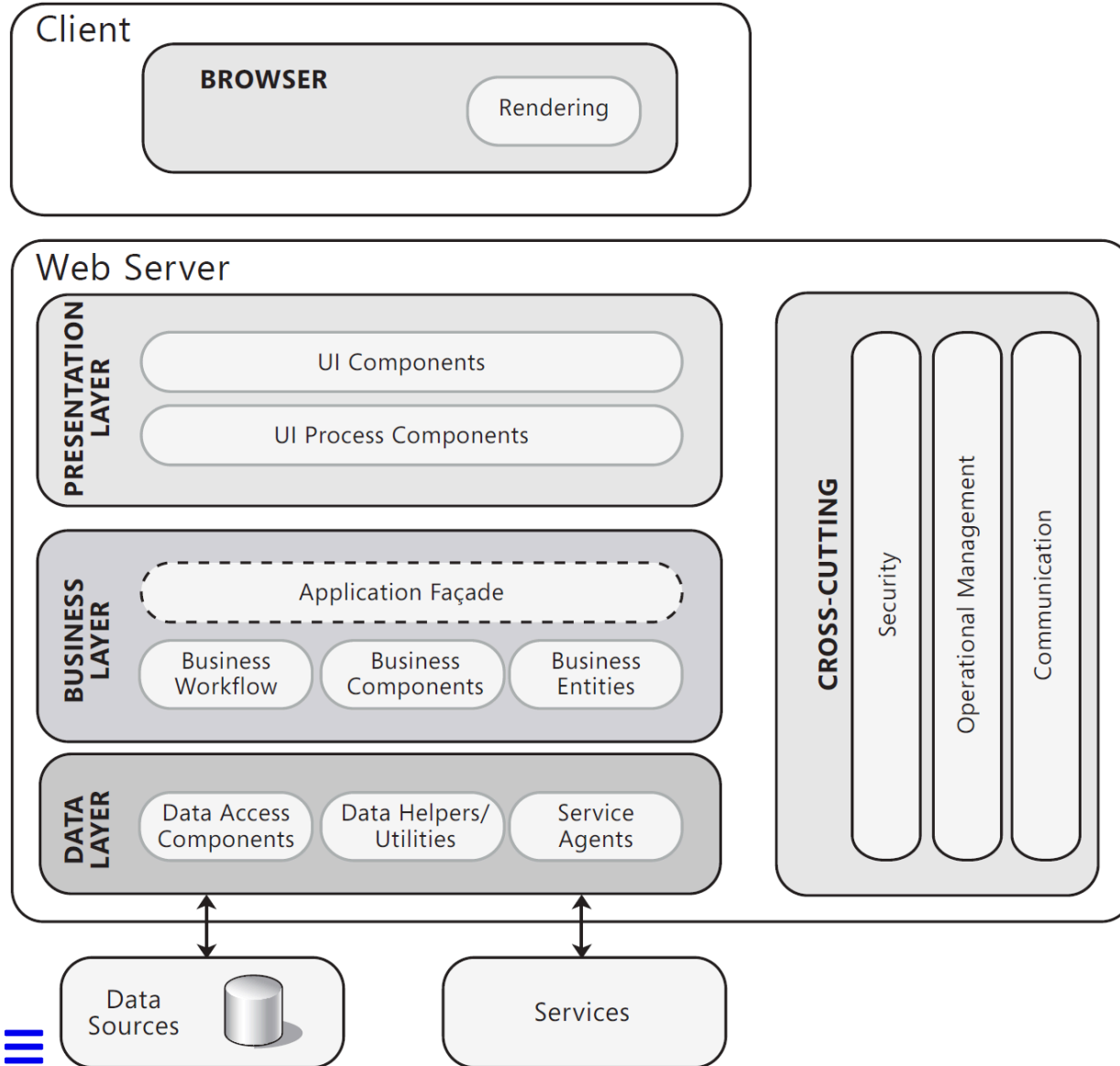
| Modell                        | Beschreibung                                                                      | Typische Technologien                                |
|-------------------------------|-----------------------------------------------------------------------------------|------------------------------------------------------|
| Server-Side Rendering (SSR)   | HTML wird auf dem Server erzeugt und an den Browser gesendet                      | PHP, Java Servlets/JSP, Django, Rails, Next.js (SSR) |
| Single-Page Application (SPA) | Browser lädt einmalig eine JS-Anwendung, die danach nur noch Daten per API abruft | React, Angular, Vue.js                               |
| Hybrid (SSR + Hydration)      | Server rendert initialen HTML, Browser übernimmt danach die Interaktion           | Next.js, Nuxt.js, SvelteKit                          |
| Static Site + API             | Statische HTML-Seiten werden vorab generiert; dynamische Daten kommen per API     | Gatsby, Hugo, Astro + REST/GraphQL                   |

Moderne Frameworks wie `Next.js` und `SvelteKit` verwischen die Grenze zwischen SSR und SPA — sie rendern serverseitig für schnelle Ladezeiten und hydrieren im Browser für Interaktivität.



# Thin Client

## Web Application



## Merkmale

- Server erzeugt vollständiges HTML
- Browser rendert — wenig Client-Logik
- jede Interaktion = neuer HTTP-Request + voller Seiten-Reload

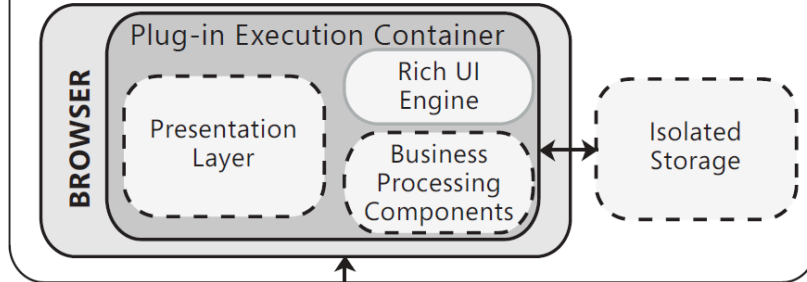
## Einsatz

- Formulareysteme, Behörden-Apps
- einfache CRUD-Anwendungen
- überall wo Client-Ressourcen knapp sind

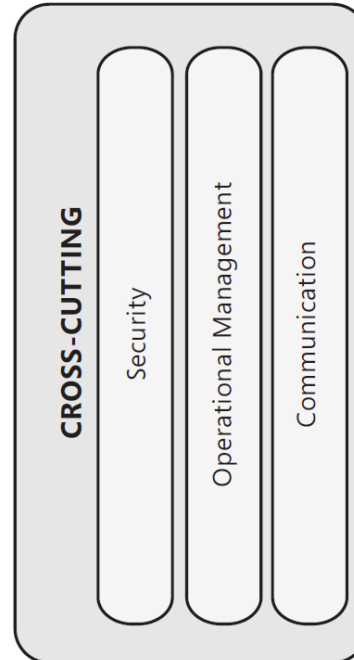
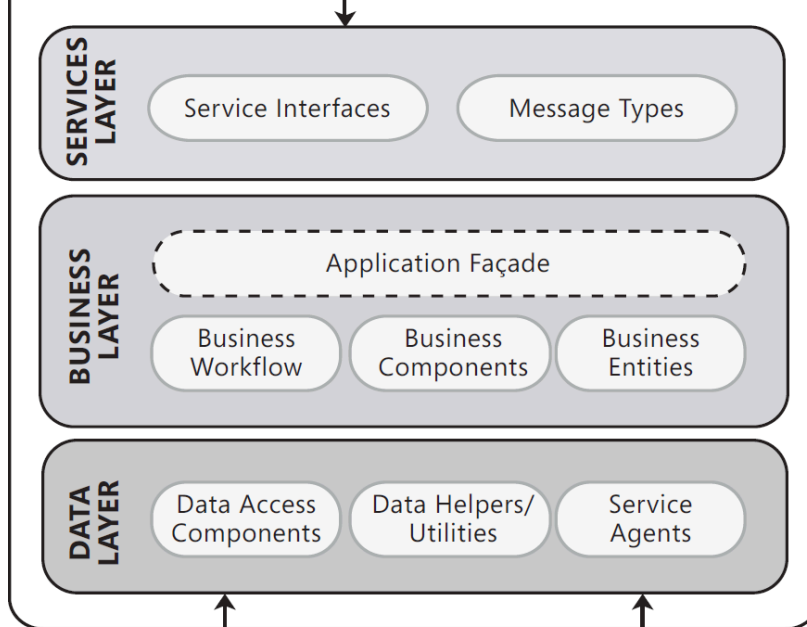
# Rich Client / SPA

## Rich Internet Application

### Client



### Web Server



## Merkmale

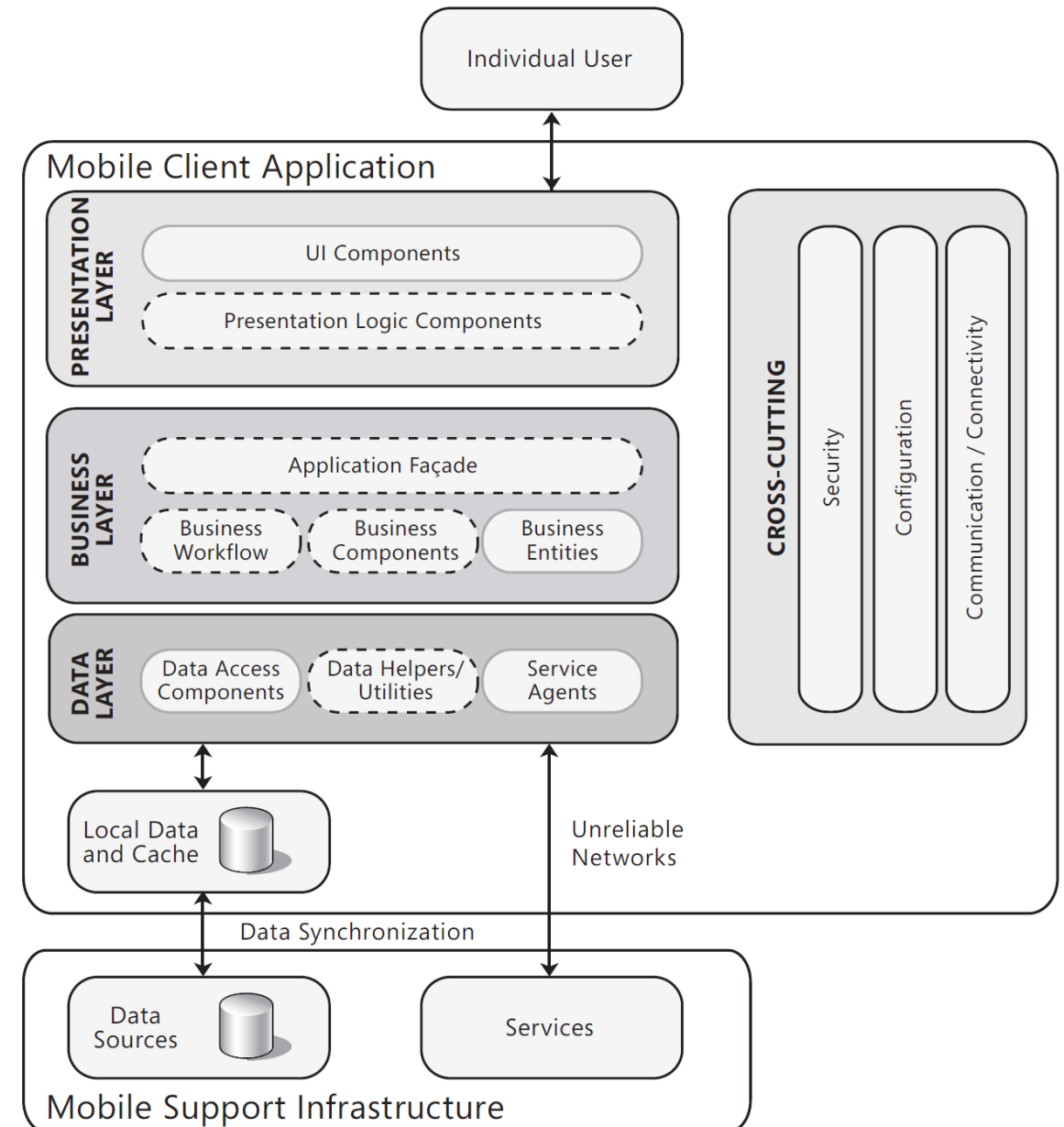
- Browser lädt einmalig eine JS-Anwendung
- API-Kommunikation per AJAX/Fetch
- flüssige Interaktion ohne Seitenneuladen

## Einsatz

- interaktive Plattformen, Dashboards
- hohe UI-Komplexität
- Offline-Fähigkeit möglich (Service Worker)

# Mobile Client

- Eigene Präsentations-, Geschäfts- und Datenschicht im Client
- Lokaler Speicher und Cache für Offline-Fähigkeit
- Datensynchronisation über unzuverlässige Netzwerke
- Cross-Cutting: Security, Configuration, Connectivity
- Zunehmend: **Progressive Web Apps (PWAs)** als Alternative zu nativen Apps



Eine moderne Web-Anwendung besteht aus weit mehr als drei Schichten. Von CDN und WAF über Load Balancer und API Gateway bis zu Message Queues und Cache — jede Komponente hat eine spezifische Aufgabe. Die Architekturentscheidung ist, welche Komponenten man braucht und wie sie zusammenspielen.

Das logische 3-Schichten-Modell beschreibt Verantwortlichkeiten; die Infrastrukturanatomie einer modernen Web-Anwendung zeigt, wie diese Schichten betrieblich aufgeteilt sind. Zwischen Nutzer und Applikation liegen DNS-Auflösung, CDN-Edge-Caches, WAF-Filterung, Load Balancing und Reverse Proxying als eigenständige Infrastrukturschicht — jede mit eigenen Caching-, Failover- und Sicherheitseigenschaften. Die Systemgrenze liegt nicht zwischen Browser und Server, sondern zwischen öffentlicher Infrastruktur und internem Applikationskern.

# Datenarchitekturen für Web-Systeme

---

**Leitfrage:** Welche Architekturmuster braucht es, um die Daten, die Web-Systeme erzeugen, systematisch zu verarbeiten und nutzbar zu machen?



# Warum Datenarchitektur Teil der Web-Architektur ist

- Moderne Web-Systeme erzeugen kontinuierlich Daten: Klicks, Transaktionen, Logs, Sensorwerte
- Produktfunktionen wie Personalisierung, Dashboards und Alerting hängen direkt von Datenpipelines ab
- Ohne explizite Datenarchitektur entstehen Ad-hoc-Lösungen, die nicht skalieren und schwer wartbar sind

Datenarchitektur und Anwendungsarchitektur entwickeln sich in modernen Systemen gemeinsam. Lambda-Architektur kombiniert Batch- und Stream-Verarbeitung: ein Batch Layer für vollständige historische Analysen, ein Speed Layer für Echtzeit-Inkrementale. Kappa-Architektur vereinfacht diesen Ansatz, indem alles als Stream behandelt wird. Die Wahl zwischen diesen Ansätzen hängt davon ab, ob Konsistenz oder Latenz die dominante Anforderung ist.

# Batch-Verarbeitung

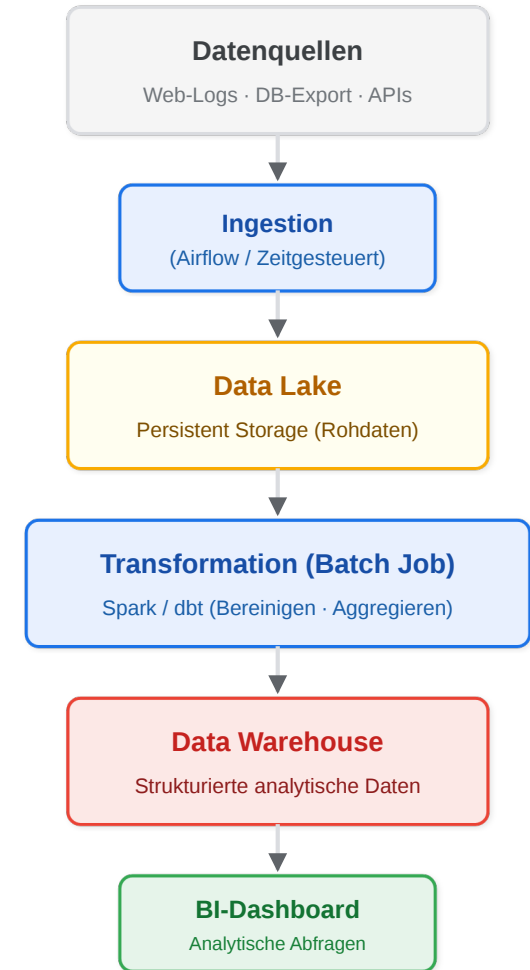
## Ablauf

1. **Quellen** (Web-Logs, DB, APIs) → **Ingestion**
2. Daten landen zeitgesteuert im **Data Lake** (Rohdaten)
3. **Transformation** (Spark) berechnet den Gesamtdatensatz
4. Ergebnisse fließen ins **Data Warehouse** für BI/Analyse
  - **Eigenschaften:** hoher Durchsatz, hohe Latenz (Min.-Std.)
  - **Gut für:** Reporting, ML-Training, historische Analysen

### Technologien:

- Apache Spark
- Apache Airflow
- Hadoop

## Batch-Verarbeitung



Batch ist stark, wenn Vollständigkeit wichtiger ist als Reaktionszeit. Das erklärt, warum viele Analyse- und ML-Pipelines weiterhin batch-orientiert bleiben.

# Stream-Verarbeitung

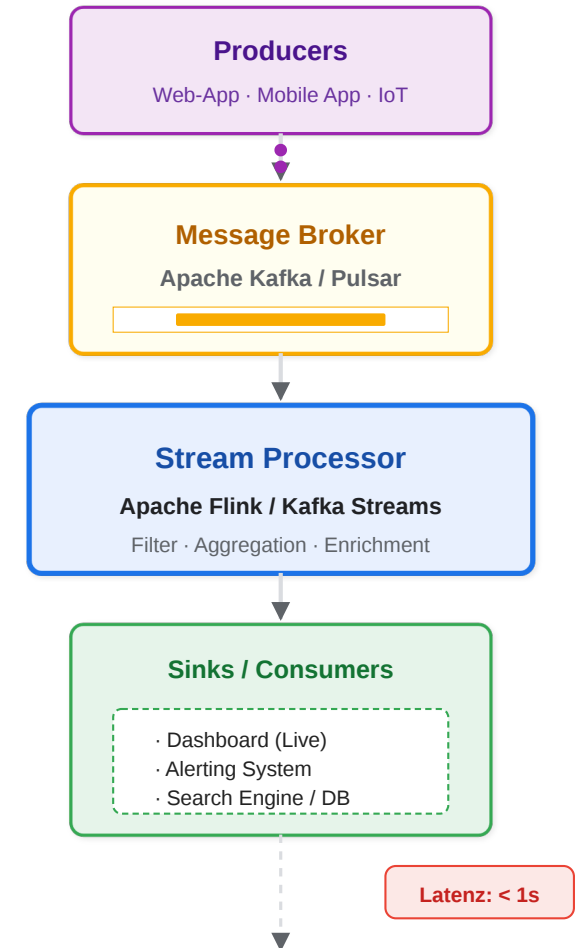
## Ablauf

1. **Producer** (Apps) senden Events an **Broker** (Kafka)
  2. **Stream Processor** (Flink) konsumiert sofort
  3. Events werden gefiltert und aggregiert
  4. **Sinks** (Dashboards, Alerts) erhalten Ergebnisse in Echtzeit
- **Eigenschaften:** niedrige Latenz (ms-s), komplexer (Order/Duplikate)
  - **Gut für:** Monitoring, Betrugserkennung, Echtzeit-Personalisierung

### Technologien:

- Apache Kafka
- Apache Flink
- Spark Streaming

## Stream-Verarbeitung



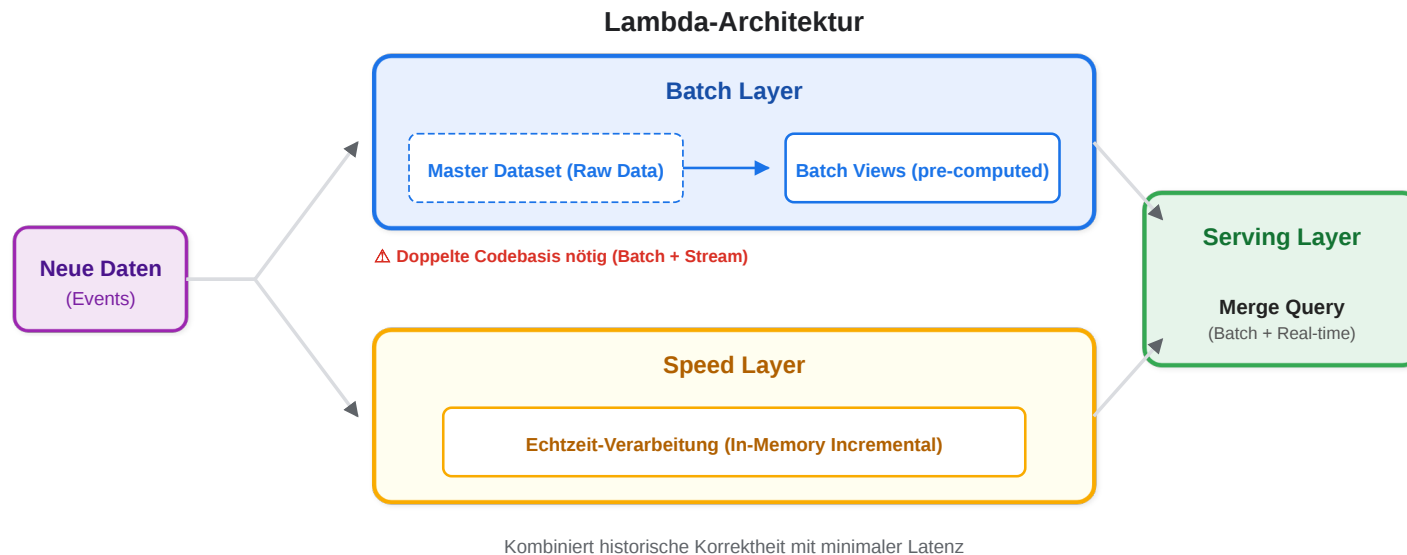
Stream-Verarbeitung verschiebt die Schwierigkeit von der Ausführung zur Robustheit. Echtzeit bedeutet auch mehr Verantwortung für Reihenfolge, Duplikate und Fehlerfälle.

# Batch vs. Stream im Vergleich

| Aspekt            | Batch                              | Stream                                   |
|-------------------|------------------------------------|------------------------------------------|
| Latenz            | Minuten bis Stunden                | Millisekunden bis Sekunden               |
| Durchsatz         | sehr hoch (große Blöcke)           | hoch (Event für Event)                   |
| Komplexität       | geringer (vollständiger Datensatz) | höher (Reihenfolge, Duplikate, Ausfälle) |
| Typischer Einsatz | Reporting, ML-Training, ETL        | Monitoring, Alerting, Echtzeit-Features  |
| Fehlerbehandlung  | Neuberechnung des gesamten Batches | Checkpoints, Exactly-once-Semantik       |

# Lambda-Architektur

**Problem:** Wie bekommt man sowohl historische Vollständigkeit als auch Echtzeit-Ergebnisse?



- **Batch Layer:** verarbeitet den vollständigen historischen Datensatz periodisch → exakte, aber verzögerte Ergebnisse
- **Speed Layer:** verarbeitet neue Events in Echtzeit → sofortige, aber möglicherweise approximierte Ergebnisse
- **Serving Layer:** kombiniert die Ergebnisse beider Pfade für Abfragen (Dashboard, API)

## Herausforderungen:

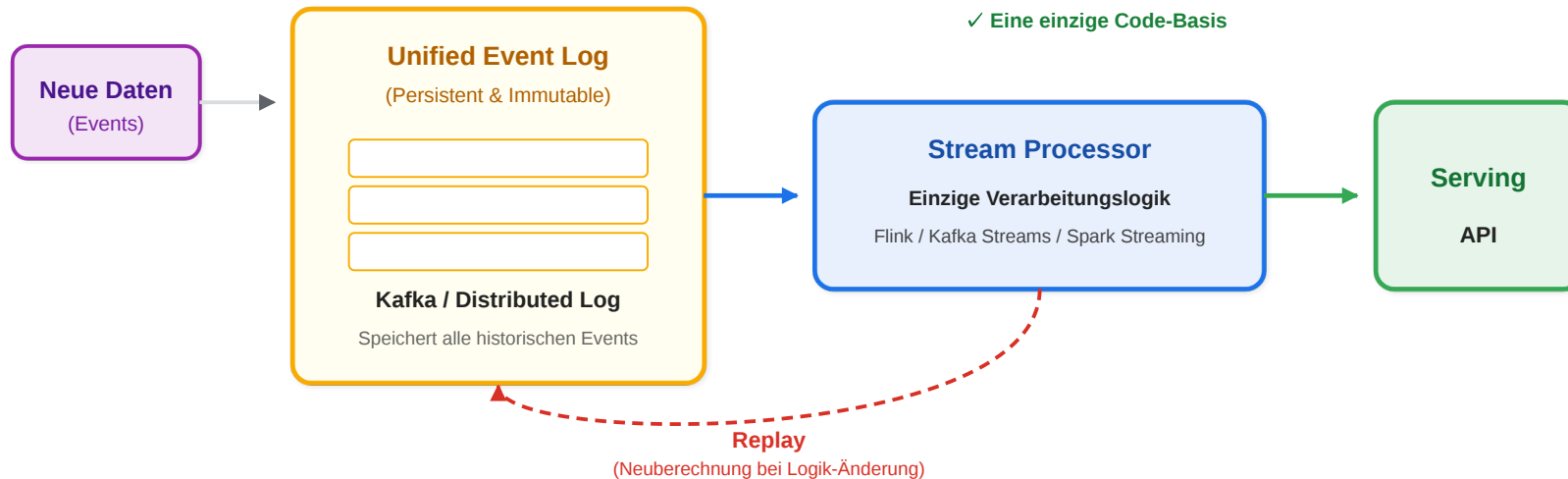
- Doppelte Codebasis nötig (Batch + Stream)
- Hoher Betriebsaufwand
- Konsistenz zwischen Pfaden schwierig



# Kappa-Architektur

**Problem:** Kann man die Komplexität der Lambda-Architektur reduzieren?

Kappa-Architektur



- **Unified Log:** alle Daten werden als Events persistent gespeichert (z.B. Kafka)
- **Stream Processing:** verarbeitet neue und historische Daten mit derselben Logik
- **Replay:** Neuberechnung bei Logik-Änderung durch erneutes Durchlaufen des Logs

## Vorteile:

- Eine einzige Codebasis für alles
- Geringerer Betriebsaufwand
- Replay ermöglicht Korrektur

## Herausforderungen:

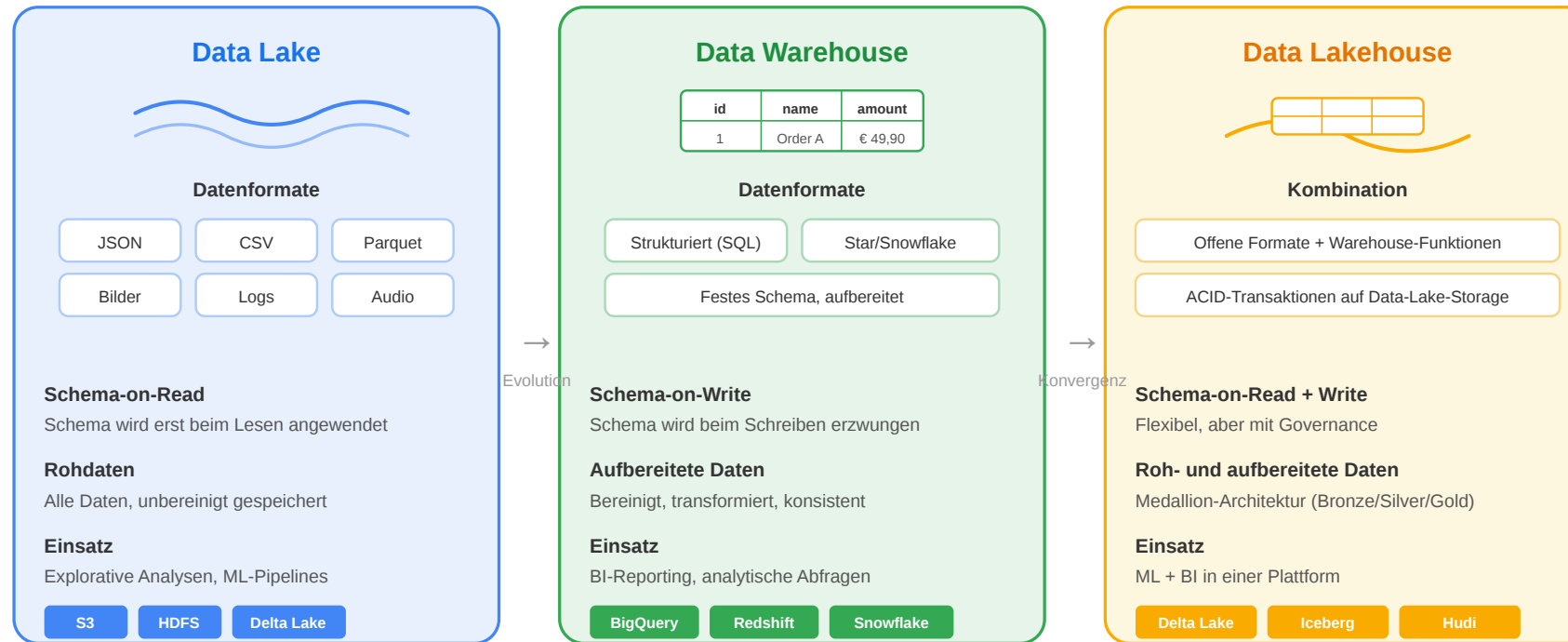
- Replay großer Logs kann teuer sein
- nicht alles rein als Stream modellierbar

# Lambda vs. Kappa im Vergleich

| Aspekt             | Lambda                            | Kappa                              |
|--------------------|-----------------------------------|------------------------------------|
| Verarbeitungspfade | zwei (Batch + Stream)             | einer (nur Stream)                 |
| Codebasis          | doppelt                           | einfach                            |
| Historische Daten  | Batch Layer                       | Replay aus Event-Log               |
| Betriebsaufwand    | hoch                              | geringer                           |
| Genauigkeit        | Batch korrigiert Stream           | Replay korrigiert                  |
| Typischer Einsatz  | Legacy-Systeme, komplexe Analysen | modernere Event-getriebene Systeme |

# Data Lake und Data Warehouse

## Data Lake vs. Data Warehouse vs. Data Lakehouse



Trend: Data Lakehouse vereint Flexibilität des Data Lake mit der Struktur des Data Warehouse

Datenarchitekturen sind keine Ergänzung zur Web-Architektur, sondern ihr integraler Bestandteil. Ob Batch oder Stream, Lambda oder Kappa, Data Lake oder Warehouse — die Wahl hängt davon ab, wie schnell, vollständig und konsistent Daten verarbeitet werden müssen.

# Schlüsselbegriffe

| Begriff               | Erklärung                                                                                                                                 |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Ingestion             | Das Einsammeln von Rohdaten aus verschiedenen Quellen in ein zentrales System                                                             |
| ETL / ELT             | <b>Extract, Transform, Load</b> — der klassische Dreischritt. Bei ELT werden Rohdaten zuerst geladen und erst im Zielsystem transformiert |
| Data Lake             | Zentraler Speicher für Rohdaten in beliebigem Format (Schema-on-Read)                                                                     |
| Data Warehouse        | Speicher für aufbereitete, strukturierte Daten mit festem Schema (Schema-on-Write)                                                        |
| Message Broker        | Infrastruktur (z.B. Kafka), die Events zwischen Producern und Consumern puffert                                                           |
| Checkpoint            | Regelmäßige Momentaufnahme des Verarbeitungszustands im Stream Processing                                                                 |
| Exactly-once-Semantik | Die Garantie, dass jedes Event genau einmal verarbeitet wird                                                                              |
| At-least-once         | Jedes Event wird mindestens einmal verarbeitet; Consumer müssen mit Duplikaten umgehen                                                    |
| Backpressure          | Mechanismus, der den Datenfluss drosselt, wenn ein Consumer langsamer verarbeitet als der Producer liefert                                |
| Replay                | Das erneute Durchlaufen eines Event-Logs; ermöglicht Neuberechnung bei Logik-Änderungen                                                   |

# Wrap-Up

- Software verwaltet physische Ressourcen (CPU, RAM, Storage, Netzwerk) — ihre Grenzen erzwingen Verteilung und damit Architekturentscheidungen.
- Architektur reduziert Komplexität, indem sie Verantwortlichkeiten explizit trennt.
- N-Tier, Event-Driven und Microservices lösen unterschiedliche Probleme — die Wahl hängt von Domäne, Last und Team ab.
- Das 3-Schichten-Modell trennt logisch; die physische Verteilung (Tiers) bringt Skalierung, aber auch Betriebskomplexität.
- Datenarchitekturen (Batch, Streaming, Lambda/Kappa) sind keine Ergänzung, sondern integraler Bestandteil moderner Web-Systeme.